scientific reports

Check for updates

OPEN

An optimized gradient boosting framework for IoT intrusion detection: a comprehensive evaluation on the CICIoT2023 dataset

Salah A. Almahaqeri[1], Mohammed Hashem Almourish[1,2], Adel A. Nasser[3,4]✉, Abed Saif Ahmed Alghawli[5], Amani A. K. Elsayed[5] & Ali N. Alhejoj[2]

The Internet of Things (IoT) generates high-volume network traffic that is heterogeneous and highly imbalanced, creating a critical need for intrusion detection systems (IDS) that remain accurate under resource-constrained edge gateways. This study presents an optimized gradient-boosting IDS framework evaluated on the CICIoT2023 benchmark. The pipeline integrates stratified undersampling to mitigate class imbalance and gain-based feature selection to remove redundant flow attributes. We evaluate the framework across three tasks: binary detection, 8-class attack-family classification, and 34-class fine-grained classification. Across all tasks, the evaluated models achieved strong detection performance with low inference latency; specifically, XGBoost attained 99.61% accuracy and a 0.995 macro-F1 score in binary detection with 0.355 µs/sample inference time. Feature selection reduced the input space from 46 to 23 dimensions, and an ablation study showed that this reduction decreased inference latency by 30–51% and training time by 33–40% while keeping accuracy variation within ± 0.014% points. These results indicate an accuracy–efficiency trade-off suitable for real-time deployment. All datasets, code, and results are publicly available to facilitate reproducibility.

Keywords IoT security, Intrusion detection, XGBoost, LightGBM, CatBoost, Feature selection, Class imbalance, CICIoT2023

Internet of Things (IoT) technologies have become foundational to contemporary digital infrastructures, enabling large-scale sensing, automation, and cyber-physical services across domains such as industrial control, healthcare, and smart cities[1,2]. However, the rapid proliferation and heterogeneity of IoT devices, protocols, and deployment environments have substantially expanded the attack surface, contributing to a sustained rise in network-borne threats[3–5]. Intrusion Detection Systems (IDSs) remain a primary defensive layer for IoT networks by learning patterns of benign and malicious behavior from traffic observations[1]. Yet, building IDSs that are simultaneously accurate and operationally deployable at the network edge remains challenging.

Two characteristics of IoT traffic are particularly problematic for real-time IDS deployment. First, IoT network traces are typically high-volume, high-dimensional, and heterogeneous, increasing feature-processing complexity and imposing non-trivial computational overhead on gateways and edge nodes[6]. Second, IoT security datasets often exhibit severe class imbalance: benign flows and a small number of dominant attack categories comprise most samples, while rare but high-impact attacks appear sparsely. Such imbalance can bias learning algorithms, distort evaluation, and degrade detection of minority attack classes that may be most critical in practice[7,8]. As a result, strong predictive performance alone is insufficient; low-latency inference and efficiency under realistic resource constraints are equally decisive for practical deployment.

Despite extensive progress in IoT IDS research, two gaps remain insufficiently addressed. Many studies prioritize accuracy and F1 performance on benchmark datasets while offering limited, rigorous analysis

[1]Department of Information Technology, Taiz University, Taiz, Yemen. [2]Department of Computer Science, Isra University, Amman, Jordan. [3]Department of Information Systems and Computer Science, Sa'adah University, Sa'adah, Yemen. [4]Department of Artificial Intelligence, Modern Specialized University, Sana'a, Yemen. [5]Department of Computer Science, College of Sciences and Humanities, Prince Sattam Bin Abdulaziz University, Al-Kharj, Riyadh Province, Saudi Arabia. ✉email: adel@saada-uni.edu.ye; dr.adel@msu.edu.ye

of the efficiency–latency trade-off that determines feasibility at the edge[8,9]. In addition, the combined effect of redundancy reduction (via feature selection) and imbalance mitigation is rarely evaluated in a unified, reproducible manner across multiple label granularities (e.g., binary detection, attack-family classification, and fine-grained attack typing), particularly with respect to minority-class robustness[10,11]. Addressing these gaps is essential for narrowing the divide between laboratory results and operational requirements in IoT environments.

To bridge this gap, we propose a latency-aware intrusion detection framework based on optimized gradient-boosting ensembles and a reproducible, sequential preprocessing pipeline. The approach is designed for large-scale traffic processing and practical edge deployment: (i) it supports memory-safe ingestion of large datasets via chunked processing; (ii) it mitigates extreme imbalance using stratified sampling to preserve class diversity while keeping training tractable; (iii) it applies gain-based feature selection to retain the most discriminative flow-level attributes and suppress redundant dimensions; and (iv) it evaluates efficient gradient-boosting models to achieve strong detection performance under stringent inference-latency constraints. By jointly optimizing data preparation, representation, and model choice, the framework targets robust detection without sacrificing real-time feasibility.

The principal contributions of this work are as follows:

1. *Unified and reproducible evaluation pipeline*: We provide a fully documented workflow on the CICIoT2023 benchmark, enabling consistent evaluation across binary (2-class), 8-class attack-family classification, and fine-grained (34-class) intrusion classification tasks.
2. *Efficiency-oriented feature selection with empirical validation*: We introduce a gain-based selection strategy that substantially reduces feature dimensionality and conduct an ablation study quantifying its impact on detection performance and inference efficiency.
3. *Latency-aware benchmarking of gradient-boosting IDS models*: We systematically evaluate leading gradient-boosting frameworks (XGBoost, LightGBM, and CatBoost), reporting standard detection metrics alongside training cost and microsecond-scale inference latency to assess practical viability for edge deployment.

The remainder of this paper is organized as follows. Section 2 reviews related work in IoT intrusion detection and efficiency-aware IDS design. Section 3 describes the CICIoT2023 dataset, threat model, and methodology. Section 4 presents experimental results, including feature-importance analysis and ablation experiments. Section 5 discusses findings, limitations, and threats to validity. Finally, Sect. 6 concludes the paper and outlines directions for future research.

Literature review

In recent years, intrusion detection in IoT networks has garnered significant research interest, and the CICIoT2023 dataset has become a widely used benchmark for assessing detection frameworks. Researchers have proposed a range of approaches, from traditional machine learning and ensemble methods to advanced deep learning, hybrid, reinforcement, and federated learning models. These studies demonstrate notable advancements in enhancing detection accuracy across binary, multi-class, and fine-grained classification tasks, while also delving into the interpretability, scalability, and real-time applicability. This review summarizes the most pertinent contributions, highlighting their strengths and limitations, to contextualize the remaining research gaps.

Neto et al.[2] present the CICIoT2023 dataset, a comprehensive resource developed using an extensive real-world IoT testbed that includes 105 diverse devices and covers 33 different cyberattacks across seven major categories. This dataset captures both benign and malicious traffic executed by compromised IoT devices targeting other IoT nodes and is available in both pcap and CSV formats with 46 extracted features. To validate the dataset's utility for intrusion detection research, the authors benchmarked several machine learning models, including Random Forest, Deep Neural Networks, Logistic Regression, Perceptron, and Adaboost, on three classification tasks: binary (benign vs. malicious), grouped (eight classes), and fine-grained multi-class (34 classes). In binary classification, Random Forest achieved an accuracy of 99.68%, whereas Deep Neural Networks reached 99.44%. For multi-class classification (eight classes), both models maintained high accuracy, with Random Forest at 99.43% and Deep Neural Networks at 99.11%. In the most challenging 34-class scenario, Random Forest achieved 99.16% accuracy, and DNN reached 98.61%. The study further reports that the F1-scores remained high across scenarios, with misclassifications concentrated in a few similar attack categories.

Abbas et al.[3] tackle the challenges of privacy, scalability, and data heterogeneity in IoT intrusion detection by proposing a federated deep neural network approach, evaluated using the CICIoT2023 dataset. Their framework employs a two-client, one-server federated architecture that enables decentralized model training while preserving data locality and privacy. This study emphasizes robust preprocessing, including feature normalization and class balancing (via SMOTE), before federated learning. Experimental results show that the proposed federated DNN approach achieves a high detection accuracy of 99.00%, validating its suitability for large-scale, privacy-sensitive IoT deployments. By moving beyond centralized paradigms, this study illustrates the potential of federated learning to provide both security and compliance with privacy requirements in real-world IoT networks.

Wang et al.[11] explore the application of deep learning techniques for network anomaly detection, utilizing the CSE-CIC-IDS2018 dataset to reflect contemporary and realistic network traffic. The authors systematically implemented and compared six different deep learning models, namely DNN, CNN, RNN, LSTM, CNN + RNN, and CNN+LSTM, for both binary and multi-class classification of network intrusions. All models demonstrated high accuracy, with multi-class classification exceeding 98.85% across the architectures. However, the study noted that hybrid models (CNN + RNN, CNN+LSTM) incurred longer inference times, making standalone DNN, RNN, and CNN models more practical for real-time IDS deployment. These findings reinforce the

importance of model efficiency, data preprocessing, and balanced evaluation when designing deep-learning-based intrusion detection systems.

Tseng et al.[7] conduct a comprehensive evaluation of deep learning models for intrusion detection in IoT networks, utilizing the large-scale and diverse CICIoT2023 dataset. This study meticulously compared seven deep learning architectures, including DNN, CNN, RNN, LSTM, CNN+LSTM, and the Transformer model, across both binary and multi-class classification tasks. Although the Transformer model trails behind the DNN and CNN+LSTM in binary classification, it excels in multi-class scenarios, achieving a peak accuracy of 99.40%. This surpasses the results reported by previous studies using the same dataset and underscores the effectiveness of attention-based architectures in managing complex and high-dimensional IoT attack data. Additionally, this study provides detailed experimental setups and benchmark comparisons, establishing a valuable reference point for future research on deep-learning-based intrusion detection in IoT environments.

Islam and Arnob[12] proposed an LSTM-based intrusion detection model specifically optimized for IoT cyber-attack detection, leveraging the extensive CICIoT2023 dataset. Their approach capitalizes on the sequential nature of IoT traffic data, with the LSTM network demonstrating strong capabilities in recognizing both the known and emerging attack patterns. Experimental results revealed that the model achieved 98.75% accuracy and a 98.59% F1 score, outperforming many prior approaches. This study highlights the relevance of adaptable deep learning models for evolving IoT threat landscapes, while also emphasizing the critical importance of using realistic, large-scale datasets for model evaluation.

Benmohamed et al.[13] propose a hybrid machine learning framework for web attack detection that integrates four classification algorithms—Gradient Boosting (GB), Multi-Layer Perceptron (MLP), Logistic Regression (LR), and K-Nearest Neighbor (KNN) with genetic algorithm-based optimization for hyper parameter tuning and feature selection. This study utilized a realistic web-attack dataset collected from the Canadian Institute 2023, encompassing a range of attack types relevant to IoT environments. In their experiments, the baseline GB model achieved the highest default accuracy (95%), precision (94%), recall (95%), and F1-score (94%) among all evaluated models. Upon applying genetic optimization, the performance improved across several models: GB maintained an accuracy of 95%, MLP improved to 86% accuracy, and LR increased to 87%. The optimized GB model consistently outperformed the others, showing superior results not only in standard metrics but also in AUC and confusion matrix analysis. These findings confirm the value of genetic algorithms in enhancing machine learning-based detection of complex and diverse web attacks, particularly in modern IoT security contexts.

He et al.[14] introduced TA-NIDS, a transferable and adaptable network intrusion detection system grounded in deep reinforcement learning, which conceptualizes intrusion detection as a sequential decision-making process. This framework allows an RL agent to develop optimal detection policies using a tailored reward function that prioritizes the precise identification of anomalous traffic. When tested on various public benchmarks, including the CICIoT2023 dataset, TA-NIDS achieved a classification accuracy of 99.22% and an F1-score of 99.15%, surpassing traditional supervised baselines while requiring only a minimal amount of labeled data for training. The study illustrates that TA-NIDS maintains a robust detection performance and strong generalization, even in dynamic, resource-constrained IoT scenarios, highlighting its practical applicability for real-time anomaly detection in complex environments.

Amouri et al.[15] propose an advanced ensemble intrusion detection framework for IoT networks by combining Kolmogorov-Arnold Networks (KANs) with the XGBoost algorithm. This ensemble utilizes KANs' adaptable and learnable activation functions of KANs to capture intricate nonlinear relationships within network traffic, whereas XGBoost offers robust and high-precision classification. Evaluated on the CICIoT2023 dataset, the hybrid system achieved a detection accuracy exceeding 99%, with precision, recall, and F1-score all above 98%, outperforming traditional multilayer perceptron (MLP) baselines across all major metrics. The results emphasize the advantages of merging symbolic function learning with gradient-boosted decision trees, providing a highly effective and interpretable solution for intrusion detection in dynamic and heterogeneous IoT environments.

Adewole et al.[16] present a transparent intrusion detection system (IDS) framework for IoT networks that combines ensemble learning with rule induction to enhance both detection performance and model explainability. This study assessed five ensemble algorithms Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost using two public datasets: CIC-IDS2017 and CICIoT2023. On the CICIoT2023 dataset, XGBoost achieved the highest performance among the tested models, with an accuracy of 98.54% and an AUC-ROC of 93.06% in the binary classification task, while the precision, recall, and F1-score all exceeded 98.5%. The framework also demonstrates competitive inference times and low false-positive/negative rates, making it suitable for real-time IoT environments. Additionally, the integration of rule induction enables the system to generate human-interpretable if–then rules that clarify the basis for each decision, addressing the critical need for transparency and stakeholder trust in practical IDS deployment.

Despite these advances, a critical analysis of the literature reveals persistent methodological and evaluative gaps, as synthesized in Table 1. First, the predominant focus remains on predictive accuracy (e.g., accuracy, F1-score), with insufficient attention to operational metrics like inference latency and computational cost—parameters critical for deployment in resource-constrained IoT edge environments. Second, many studies evaluate performance at only a single classification granularity (e.g., solely binary or multi-class), leaving the comparative trade-offs between coarse and fine-grained detection strategies underexplored. Third, while techniques like feature selection and class balancing are occasionally mentioned, their combined impact on both model efficiency and minority-class detection is rarely quantified within a reproducible, end-to-end pipeline.

Furthermore, it is essential to delineate the specific notion of scalability addressed in this domain. While recent frameworks like RACEMAN[17] and ASRL[18] effectively tackle scalability across heterogeneous online social network (OSN) platforms—addressing challenges of cross-platform behavioral adaptation and policy learning—the scalability imperative in IoT intrusion detection is fundamentally different. Our work addresses

Ref.	Classification	DL/ML method	Accuracy	Inference (µs/sample)
[2]	Binary, Multi-class	Random Forest, Deep Neural Networks (DNN), others	99.68% (RF, Binary), 99.44% (DNN, Binary), 99.43% (RF, 8 classes), 99.16% (RF, 34 classes),	-
[3]	Binary	Federated (DNN)	99.00%	-
[11]	Binary, Multi-class	DNN, CNN, RNN, LSTM, CNN+LSTM, CNN + RNN	> 98.85% (Multi-class)	LSTM: 3.451 ms, CNN+LSTM: 4.31 ms
[7]	Binary, Multi-class	Transformer-based DNN, DNN, CNN+LSTM, others	99.57% (Binary), 99.40% (Multi-class)	≈ 5 µs/sample (Transformer)
[12]	Multi-class	LSTM	98.75%	-
[13]	Binary, Multi-class	Gradient Boosting (GB), MLP, Logistic Regression, KNN, Genetic Optimization	95.00% (GB, optimized)	-
[14]	Binary, Multi-class	Deep Reinforcement Learning (TA-NIDS)	99.22% (Binary, CICIoT2023), F1: 99.15%	-
[15]	Binary, Multi-class	KAN + XGBoost Ensemble	> 99.0% (Accuracy), F1, Precision, Recall > 98% (Multi-class)	-
[16]	Binary	XGBoost, LGBM, CatBoost, RF, AdaBoost	98.54% (XGBoost, Binary)	-

Table 1. Comparative evaluation of classification models on CICIoT2023 dataset.

scalability within high-volume, homogeneous IoT network traffic, focusing on the computational challenge of sustaining low-latency, high-throughput inference directly at the resource-constrained network edge. This involves architectural optimizations such as chunked stream processing, discriminative feature selection, and model efficiency tuning, which are orthogonal to the OSN-specific challenges of cross-platform heterogeneity and adversarial behavior modeling.

Methods

This section details the experimental protocol used to construct and evaluate the proposed intrusion detection framework on the CICIoT2023 dataset. The full processing and evaluation workflow is summarized in Fig. 1, which provides a reproducible, end-to-end pipeline for generating three classification tasks: binary (2-class), 8-class attack-family classification, and fine-grained (34-class) and benchmarking multiple learning algorithms under identical preprocessing, tuning, and testing conditions. To ensure unbiased generalization estimates, all steps that could learn from the data (feature selection, scaling statistics, and hyperparameter tuning) were performed exclusively on the training partition, and the held-out test set was used once for final reporting.

Study design, data source, and selection criteria

Study design

This work is an empirical benchmarking study based on supervised classification at three detection granularities. The primary outcomes are classification effectiveness (accuracy and F1-based metrics) and deployment-relevant efficiency (per-sample inference latency).

Data source

All experiments used the CICIoT2023 dataset[2], which contains over 46 million labeled network-flow records collected from a real IoT testbed comprising 105 heterogeneous devices. Each record is described by 46 numerical flow features and annotated with ground-truth labels spanning benign traffic and 33 distinct attack types grouped into major families (e.g., DDoS/DoS, reconnaissance, web-based attacks, brute force, spoofing, and Mirai-related activities).

Inclusion and exclusion criteria

We included only the flow records and labels provided by CICIoT2023[2] and operated strictly on the numeric flow-feature representation (payload-agnostic). Records failing basic integrity checks required for model training (e.g., non-finite feature values) were excluded during preprocessing. No payload inspection or application-layer content was used, and no data outside CICIoT2023 was incorporated. Because CICIoT2023 is extremely imbalanced and very large, directly training on the full dataset is computationally expensive and can bias learning toward majority classes. We therefore used stratified undersampling to construct task-specific datasets that remain statistically representative while preserving minority, security-critical attack classes. The resulting class distributions before and after undersampling are shown in Figs. 2, 3 and 4.

Threat model and deployment scenario

Deployment scenario

We target a typical IoT security architecture in which traffic from resource-constrained devices aggregates at a gateway or edge node (e.g., smart gateway, industrial edge appliance, campus switch). The IDS operates on flow-level statistical descriptors consistent with CICIoT2023, enabling near-real-time monitoring without reliance on packet payloads. Models follow an offline training/online inference paradigm: training and validation are conducted in a controlled environment, while the deployed IDS performs streaming flow classification at the aggregation point.

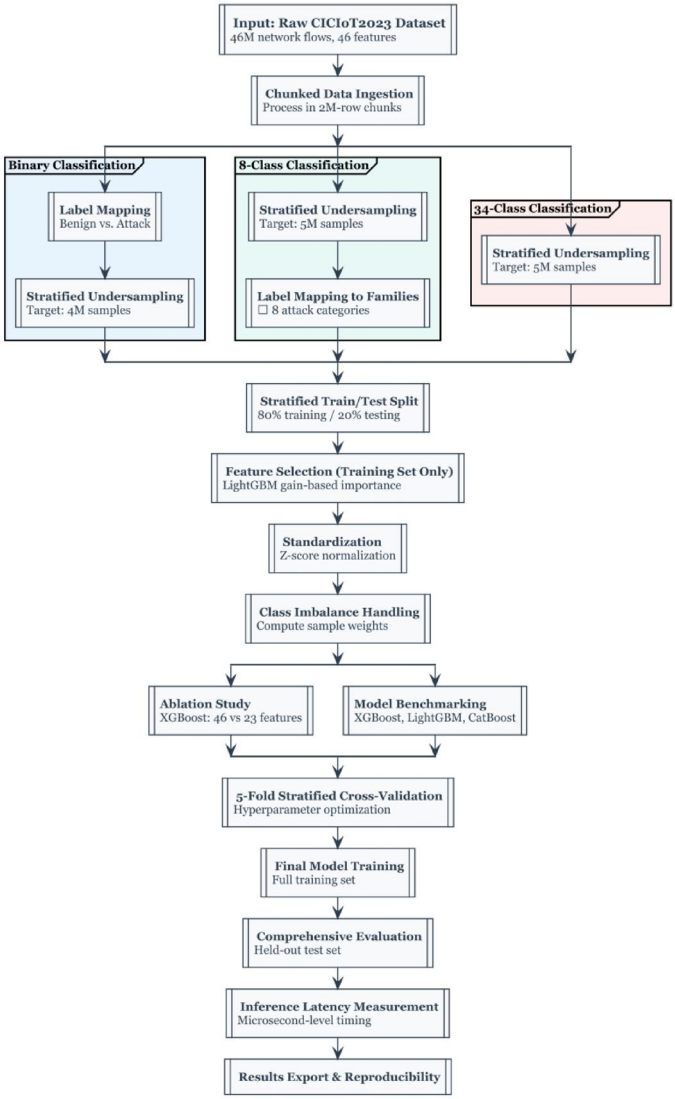


Fig. 1. Proposed framework workflow.

Adversary model

The adversary aims to compromise IoT services and devices through attacks observable at the gateway (e.g., reconnaissance scans, brute-force attempts, DoS/DDoS floods, and application-layer exploits). The adversary may attempt evasion by shaping traffic statistics (e.g., adjusting rates or inter-arrival characteristics) to resemble benign patterns. We assume the attacker cannot directly compromise the IDS sensor or manipulate the feature extraction logic.

Scope

Two design choices bound the study: (i) trusted feature extraction—attacks targeting the sensing layer are out of scope; and (ii) payload-agnostic detection—only flow metadata are used, supporting privacy-preserving operation and compatibility with encrypted traffic.

Data preprocessing and task construction

Given the dataset scale and imbalance, we implemented a structured preprocessing pipeline comprising chunk-wise ingestion, label mapping, stratified undersampling, leakage-safe splitting, and feature standardization[19].

Chunk-wise ingestion

To ensure memory-safe processing of ~46 million records, CICIoT2023 CSV files were read sequentially in fixed-size chunks of 2 million rows. During ingestion, label frequencies were aggregated to quantify imbalance and to parameterize the subsequent stratified sampling strategy (Fig. 1).

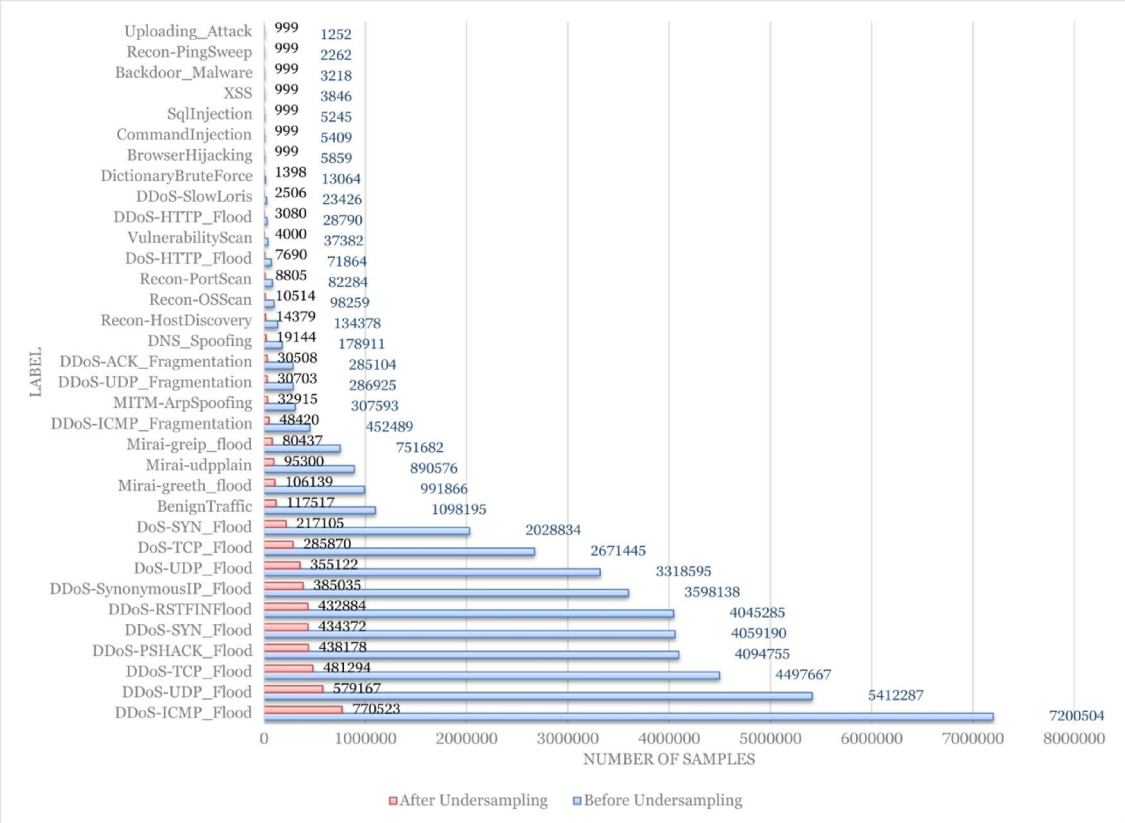


Fig. 2. Original vs. undersampled class distribution for the 34-class CICIoT2023 dataset.

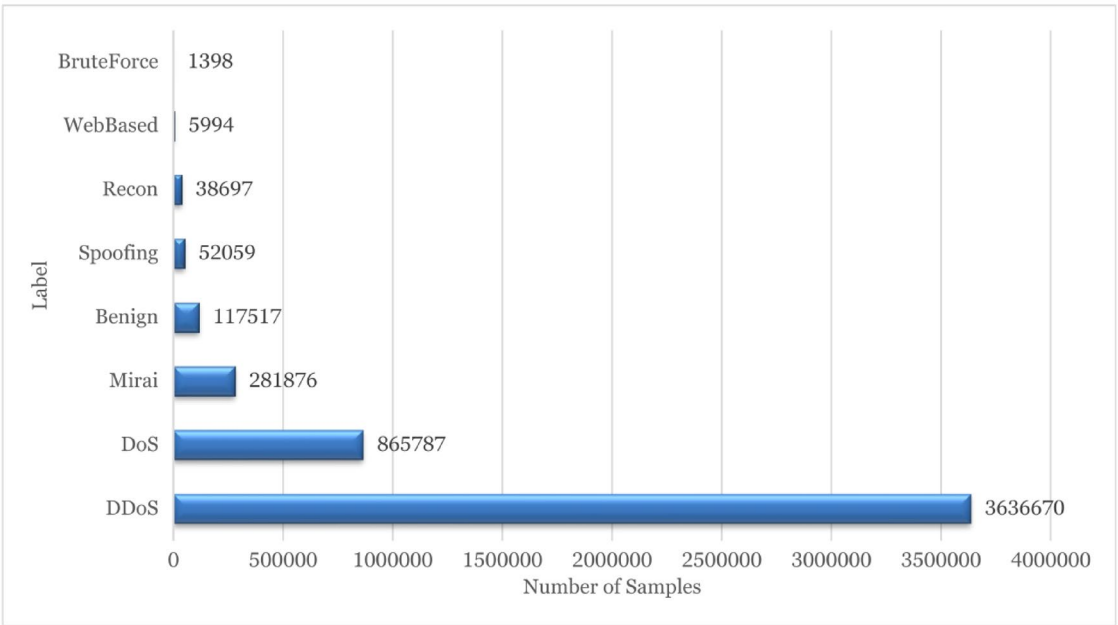


Fig. 3. Undersampled class distribution for the 8-class CICIoT2023 dataset.

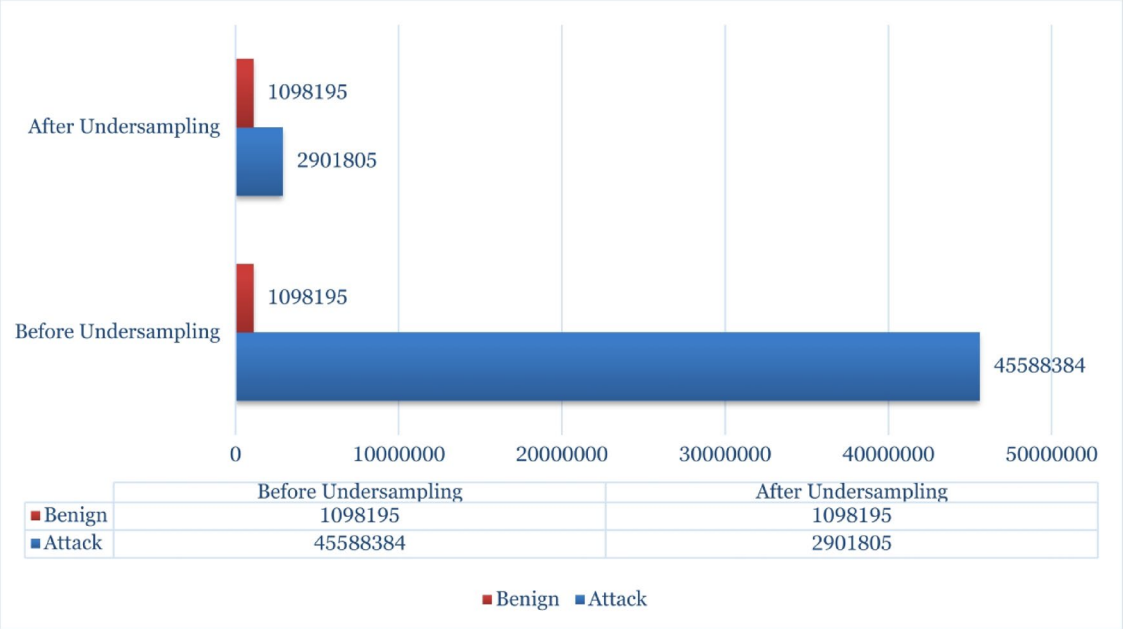


Fig. 4. Original vs. undersampled binary class distribution of the CICIoT2023 dataset.

Label mapping and stratified undersampling
Three parallel dataset variants were constructed (Fig. 1):

- *Binary classification (2-class)*: all labels were mapped to Benign or Attack. All benign flows (1,098,195) were retained, and 2,901,805 attack flows were randomly sampled to form a 4,000,000-instance working set (Fig. 4).
- *Fine-grained classification (34-class)*: stratified sampling reduced the dataset to 5,000000 flows while preserving the original 34-class label structure. A minimum of 1000 samples per class was enforced to prevent removal of rare but security-relevant classes and to support reliable per-class evaluation (Fig. 2).
- *Coarse-grained classification (8-class)*: original attack labels were aggregated into eight functional families (e.g., DDoS variants → "DDoS", web-oriented attacks → "WebBased"). Stratified sampling produced 5,000000 flows with minimum representation constraints at the family level (Fig. 3).

The stratified undersampling allocation for each class c_k is formalized as:

$$n_k^{\text{target}} = \max(M_{\min}, \lfloor \frac{n_k}{N_{\text{orig}}} \times N_{\text{target}} \rfloor) \tag{1}$$

Where $\mathcal{C} = \{c_1, \ldots, c_K\}$ is the class set, n_k is the original count of class c_k, $N_{\text{orig}} = \sum_k n_k$ is the total sample count, N_{target} is the post-sampling dataset size (4 M for binary and 5 M for multi-class), and $M_{\min}$ is the minimum retained samples per class (1,000 for multi-class tasks). These target sizes (4–5 million) were selected to balance computational tractability for cross-validation and gradient-boosting training with statistical reliability through large held-out test sets, while the minimum-per-class constraint prevents rare but security-critical attack types from being eliminated and supports stable per-class evaluation.

Train–test splitting and study protocol
Each dataset variant was divided into 80% training and 20% testing using stratified sampling to preserve class proportions. This split provides a robust compromise between learning capacity and evaluation reliability at the considered scale: with 4–5 million samples after undersampling, the training set (≈3.2–4.0 M) is sufficiently large to stabilize model estimation and support five-fold cross-validation, while the test set (≈0.8–1.0 M) yields low-variance estimates of overall and per-class performance. To prevent data leakage and optimistic bias, all data-dependent steps (feature selection, normalization parameter estimation, and hyperparameter tuning) were conducted exclusively on the training partition, and the test partition was used once for final evaluation.

Label encoding and feature standardization
Labels were encoded as integer indices. To improve numerical stability and facilitate optimization, particularly for linear and distance-based baselines, all features were standardized using training-only statistics[20]:

$$x_{ij}^{\text{std}} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{2}$$

Where x_{ij} is feature j of sample i, and μ_j, σ_j are computed only on the training set and then applied to both training and test sets.

Embedded feature selection

To reduce computational cost and limit overfitting, we adopted an embedded feature selection approach using LightGBM gain-based importance[21]. Feature gain is defined as the cumulative reduction in objective loss achieved by splits on feature f_j:

$$Gain\,(f_j) = \sum_{s \in \mathcal{S}(f_j)} \Delta\,\mathcal{L}_s \tag{3}$$

A LightGBM model was trained on a 200,000-sample subset of the training partition, after which gain values were computed for all 46 features. We then applied a median-threshold rule:

$$\mathcal{F}_{\text{selected}} = \{f_j \mid \text{Gain}\,(f_j) > \text{Median}\,(\{\,\text{Gain}\,(f_k)\,\}_{k=1}^{d})\} \tag{4}$$

yielding a compact subset of 23 features. The retained feature set is detailed in Table 2, categorized by statistical function.

As visualized in Table 2, the retained features are predominantly statistical flow descriptors (e.g., IAT, Magnitude, Variance) and specific TCP control flags. This selection process revealed significant redundancy in the original dataset. Specifically, boolean protocol flags (such as HTTP, HTTPS, DNS, SSH, and LLC) were consistently ranked with near-zero information gain and were subsequently pruned.

The exclusion of these boolean attributes is justified by their limited discriminative power in the context of IoT traffic:

1. *Protocol Redundancy*: The fundamental transport layer information is already encoded in the integer-based *Protocol Type* feature.
2. *Behavioral Dominance*: In automated IoT environments, both benign and malicious flows frequently utilize standard protocols (e.g., HTTP, DNS). Consequently, the presence of a protocol flag (e.g., HTTP = 1) is not inherently anomalous. The discriminative signal lies instead in the *statistical deviation* from expected machine-to-machine patterns—such as abnormal *Rates*, *Inter-Arrival Times (IAT)*, or packet size variations (*Magnitude*)—which are fully captured by the retained feature set.

Gain-based selection is well suited to high-performance tabular learning because it reflects each feature's direct contribution to reducing the optimized objective within tree ensembles. Unlike filter-based methods (e.g., Pearson correlation) that capture only linear dependencies, or wrapper methods that are computationally prohibitive for datasets of this magnitude, the LightGBM gain-based approach provides a critically comparative advantage. Furthermore, it preserves the semantic interpretability of the original network attributes—unlike dimensionality reduction techniques such as PCA, which obscure feature meaning by transforming the feature space. By directly evaluating the objective function's reduction, this approach effectively isolates the non-linear, multidimensional feature interactions that are most discriminative for complex IoT attack vectors. Feature selection was performed using only the training partition to preserve evaluation validity, and the resulting dimensionality reduction lowers inference-time cost by reducing the number of input features evaluated at prediction time. To quantify the isolated effect of this dimensionality reduction on both accuracy and efficiency, we additionally conducted an ablation study (Sect. 4.6) in which XGBoost was trained and evaluated using either all 46 original features or only the selected 23-feature subset under otherwise identical data construction, tuning, and testing conditions.

Model pipelines and training strategy

Binary classification pipeline

For binary intrusion detection, we benchmarked a broad suite of models to cover strong tabular learners and classical baselines:

Feature category	Selected features	Description & rationale
Traffic volume & velocity	flow_duration, Rate, Srate, Drate, Tot sum, Number, Tot size	Capture volumetric anomalies (e.g., flooding attacks) by measuring packet transmission speeds and total data volume per flow.
Inter-arrival dynamics	IAT (Inter-Arrival Time)	Measures the time gap between packets; critical for distinguishing automated bot traffic (uniform IAT) from human traffic.
Packet length statistics	Header_Length, Min, Max, AVG, Std	Descriptive statistics of packet sizes. Essential for detecting attacks with fixed packet sizes (e.g., certain DDoS payloads).
Distributional metrics	Magnitude, Radius, Covariance, Variance, Weight	Complex aggregated metrics that represent the statistical distribution and correlation of incoming/outgoing traffic, providing a robust fingerprint against evasion.
TCP/IP flags	syn_flag_number, rst_flag_number, psh_flag_number, ack_flag_number, rst_count	Capture state-exhaustion attacks (e.g., SYN Flood) and scanning activities (e.g., RST/ACK scans) by tracking flag frequency.
Protocol identification	Protocol Type	Integer encoding of the transport protocol (TCP, UDP, ICMP), rendering boolean protocol flags redundant.

Table 2. The subset of 23 features retained by the gain-based selection algorithm.

- *Gradient boosting*: XGBoost, LightGBM, CatBoost, NGBoost.
- *Ensembles and linear models*: ExtraTrees, Logistic Regression, AdaBoost.
- *Classical baselines*: Decision Tree, Random Forest, Linear SVM, Gaussian Naive Bayes.

To account for any residual imbalance, class weights were computed as[22]:

$$w_i = \frac{N}{k \cdot n_i} \tag{5}$$

where N is the number of training samples, $k = 2$, n_i is the sample count of class i.

Hyperparameters were optimized via grid search with five-fold stratified cross-validation on the training set[23], using macro F1-score as the selection objective. Final models were retrained on the full training split using the selected configuration and evaluated on the held-out test set. This model suite combines state-of-the-art boosting methods with widely used baselines to quantify the accuracy–latency trade-off and to reflect realistic deployment choices in IoT IDS settings.

Multi-class pipeline (34-class and 8-class)
For multi-class classification, we benchmarked three scalable gradient-boosting frameworks known to perform strongly on tabular data and to support native multiclass objectives:

- XGBoost
- LightGBM
- CatBoost

Class imbalance was mitigated via inverse-frequency weights:

$$w_k = \frac{N}{K \cdot n_k} \tag{6}$$

Where N is the number of training samples, K is the number of classes, and n_k is the number of samples in class k. Early stopping was used where supported, halting training if validation loss did not improve for 20–30 iterations. Gradient-boosting frameworks were emphasized because they are consistently strong for large-scale tabular learning, provide native mechanisms for multiclass objectives and imbalance handling, and remain computationally practical at the 34-class scale used in this study.

Evaluation metrics and inference latency

Performance was evaluated using standard supervised classification metrics[24,25]. Accuracy is defined as[26]:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{7}$$

Per-class precision, recall, and F1-score were computed as[27,28]:

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}, \quad \text{Recall}_i = \frac{TP_i}{TP_i + FN_i} \tag{8}$$

$$\text{F1}_i = 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \tag{9}$$

Inference latency measurement
Preprocessing (integrity checks, encoding/scaling, and feature selection) was executed on the CPU, while model training employed GPU acceleration when supported. Reported inference times (µs/sample) correspond to model-only prediction latency on CPU using the held-out test set after preprocessing; data loading and preprocessing time were excluded. Latency was estimated from batch prediction over the full held-out test set and reported as elapsed wall-clock time divided by the number of samples, which reduces timing noise and yields stable per-sample estimates under a fixed software/hardware configuration. Extremely small values for optimized models should be interpreted comparatively under the same protocol rather than as device-specific absolute timings[23,29].

Algorithmic steps (workflow summary)

To address reproducibility and clarify the operational flow of the proposed method (Fig. 1), the complete procedure is summarized below.

Input:

- D_{raw}: Stream of raw CICIoT2023 CSV files.
- N_{target}: Target dataset size (4×10^6 for binary, 5×10^6 for multi-class).
- M_{min}: Minimum samples per class threshold ($M_{min} = 1{,}000$).
- F_{all}: Original feature set ($| F_{all} | = 46$).

Output:

- M^*: Optimized Gradient Boosting Model.
- F_{opt}: Selected feature subset ($| F_{opt} | = 23$).
- P_{test}: Performance metrics on held-out test set.

1. Phase I: Chunked Ingestion & Class Quantification

- **Initialize** global class counts $C_{global} \leftarrow \{0\}^K$.
- **For each** chunk $B_i \in D_{raw}$ **do**:
  - Update $C_{global} \leftarrow C_{global} + \text{CountLabels}(B_i)$.
- **End For**
- Calculate total samples $N_{total} \leftarrow \sum C_{global}$.

2. Phase II: Stratified Target Allocation

- **For each** class $k \in \{1, \ldots, K\}$ **do**:
  - Calculate proportional allocation: $n_k^{prop} \leftarrow \lfloor \frac{C_{global}[k]}{N_{total}} \times N_{target} \rfloor$.
  - Apply minimum threshold constraint:
    $$n_k^{target} \leftarrow \max(M_{min}, n_k^{prop})$$
  - Set remaining quota $r_k \leftarrow n_k^{target}$.
- **End For**

3. Phase III: Stream Sampling & Dataset Construction

- **Initialize** balanced dataset $D_{balanced} \leftarrow \emptyset$.
- **For each** $B_i \in D_{raw}$ **do**:
  - **If** $\sum r_k = 0$ **then** Break.
  - **For each** class k present in B_i **do**:
    - Identify indices Idx_k in B_i.
    - Determine sample size: $s \leftarrow \min(r_k, | Idx_k |)$.
    - Randomly select $S_k \subset Idx_k$ where $| S_k | = s$.
    - $D_{balanced} \leftarrow D_{balanced} \cup S_k$.
    - Update quota: $r_k \leftarrow r_k - s$.
  - **End For**
- **End For**

4. Phase IV: Leakage-Free Splitting & Selection

- Split $D_{balanced}$ into Training (D_{train}) and Testing (D_{test}) sets ($80/20$ stratified).
- **Feature Selection (on D_{train} only):**
  - Sample subset $D_{sub} \subset D_{train}$ ($N = 200{,}000$).
  - Train LightGBM estimator E on D_{sub}.
  - Compute Gain Importance for each feature $f \in F_{all}$ (Eq. 3):
    $$Gain(f) = \sum_{t \in \text{Trees}} \sum_{n \in \text{Nodes}(t)} I(v_n = f) \cdot \Delta L_n$$
  - Determine threshold: $\tau \leftarrow \text{Median}(\{Gain(f) \mid f \in F_{all}\})$.
  - Select features: $F_{opt} \leftarrow \{f \in F_{all} \mid Gain(f) > \tau\}$.

5. Phase V: Model Training & Evaluation

- **Standardization:** Fit μ, σ on $D_{train}[F_{opt}]$; transform D_{train} and D_{test}.
- **Training:** Compute weights (Eq. 6) and train classifier M^* on D_{train} with 5-fold CV.
- **Evaluation:** Report metrics and inference latency on D_{test}.
- **Return** M^*, F_{opt}, P_{test}.

Algorithm 1. Optimized gradient boosting IDS pipeline with stratified undersampling and gain-based feature selection.

Project	Properties
OS	Windows 11 (build 10.0.26100.1)
CPU	Intel(R) Core(TM) i9-14900HX
GPU	NVIDIA GeForce RTX 4060 (8 GB VRAM)
Memory	31.71 GB RAM
Disk	1T SSD
Python	3.11.2
CUDA Version	v12.2.91, cuda_12.2.r12.2
Frameworks	PyTorch: 2.7.1 + cu128 / XGBoost: 3.0.2 / NumPy: 2.1.2 / LightGBM: 4.6.0 / Scikit-learn: 1.6.1 / Pandas: 2.3.0

Table 3. Equipment specifications and environment settings.

Model	Accuracy	Macro-F1	Weighted-F1	CV Macro-F1	Inf (µs)	Train (s)
XGBoost	0.9961	0.9952	0.9961	0.9950 ± 0.0001	0.355	19.9
NGBoost	0.9958	0.9947	0.9958	0.9947 ± 0.0001	60.810	2599.0
CatBoost	0.9958	0.9947	0.9958	0.9944 ± 0.0001	0.513	91.2
Random Forest	0.9955	0.9943	0.9955	0.9942 ± 0.0002	11.580	97.3
LightGBM	0.9950	0.9937	0.9950	0.9926 ± 0.0009	0.713	11.0
Decision Tree	0.9948	0.9936	0.9949	0.9935 ± 0.0001	0.077	27.7
ExtraTrees	0.9891	0.9865	0.9892	0.9866 ± 0.0002	10.321	282.0
Logistic Regression	0.9819	0.9775	0.9820	0.9776 ± 0.0001	0.029	13.5
Linear SVM (SGD)	0.9818	0.9775	0.9819	0.9775 ± 0.0003	1.054	26.7
AdaBoost	0.9812	0.9767	0.9813	0.9765 ± 0.0002	2.864	192.3
Gaussian NB	0.9666	0.9596	0.9672	0.9593 ± 0.0002	0.382	0.961

Table 4. Binary classification performance (test set) with 5-fold CV stability and inference latency. *Metrics are reported on the held-out test set; CV values are 5-fold stratified Macro-F1 (mean ± std). Inference latency is average per-sample prediction time. *.

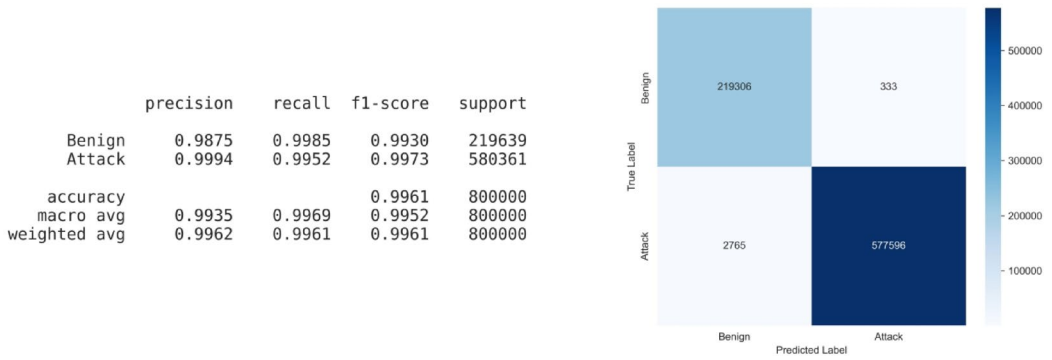


Fig. 5. Classification report and confusion matrix of XGBoost for binary classification.

Experimental results

Experimental setup

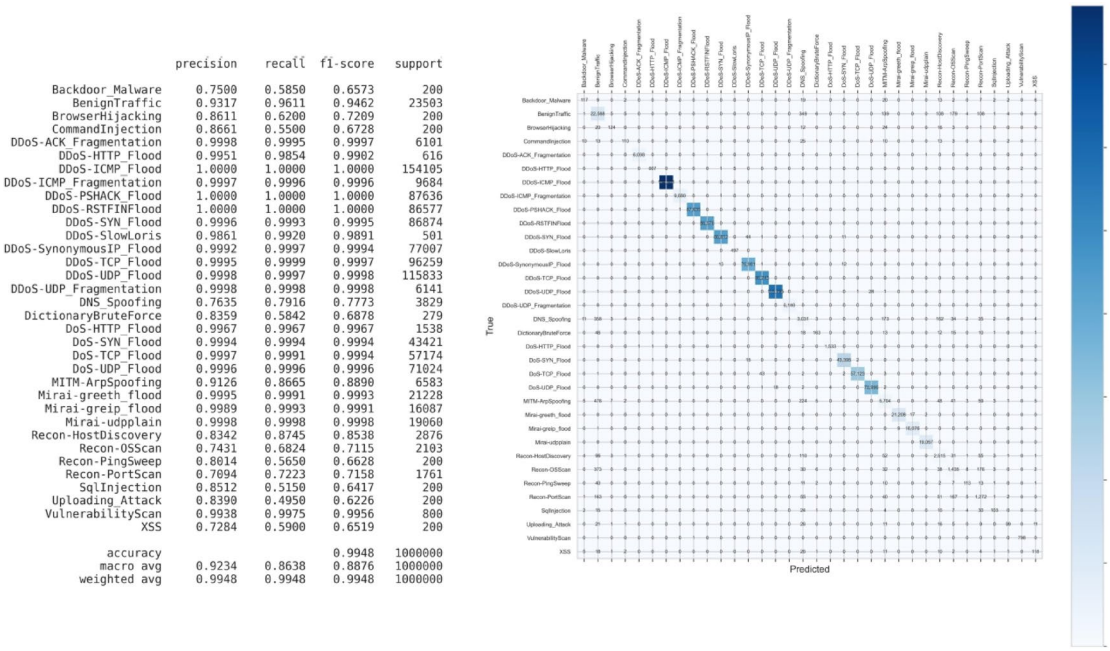
All experiments were conducted on a workstation with the specifications detailed in Table 3. The training process utilized Python 3.11.2, incorporating GPU-accelerated libraries such as PyTorch (v2.7.1 + cu128), XGBoost (v3.0.2), LightGBM (v4.6.0), and additional frameworks including NumPy (v2.1.2), Scikit-learn (v1.6.1), and Pandas (v2.3.0). An NVIDIA GeForce RTX 4060 (8 GB VRAM) provided the necessary GPU support, running on CUDA v12.2.91. The hyperparameters for all models were determined through a grid search with stratified 5-fold cross-validation, focusing on optimizing the macro F1-score on the training set.

Binary classification results

Table 4 reports held-out test performance for the binary CICIoT2023 task (benign vs. attack), together with 5-fold stratified cross-validation (CV) macro-F1 (mean ± std), average per-sample inference latency, and training time under the experimental setup in Table 3. Across the evaluated models, the highest test accuracy

Model	Accuracy	Macro-F1	Weighted-F1	CV Macro-F1	Inf (µs)	Train (s)
XGBoost	0.9948	0.8876	0.9948	0.8883 ± 0.0013	3.528	433.9
CatBoost	0.9873	0.7405	0.9883	0.7438 ± 0.0037	1.121	248.3
LightGBM	0.9879	0.7453	0.9896	0.7471 ± 0.0060	3.385	214.04

Table 5. 34-class performance (test set) with 5-fold CV stability and inference latency.



	precision	recall	f1-score	support
Backdoor_Malware	0.7500	0.5850	0.6573	200
BenignTraffic	0.9317	0.9611	0.9462	23503
BrowserHijacking	0.8611	0.6200	0.7209	200
CommandInjection	0.8661	0.5500	0.6728	200
DDoS-ACK_Fragmentation	0.9998	0.9995	0.9997	6101
DDoS-HTTP_Flood	0.9951	0.9854	0.9902	616
DDoS-ICMP_Flood	1.0000	1.0000	1.0000	154105
DDoS-ICMP_Fragmentation	0.9997	0.9996	0.9996	9684
DDoS-PSHACK_Flood	1.0000	1.0000	1.0000	87636
DDoS-RSTFINFlood	1.0000	1.0000	1.0000	86577
DDoS-SYN_Flood	0.9996	0.9993	0.9995	86874
DDoS-SlowLoris	0.9861	0.9920	0.9891	501
DDoS-SynonymousIP_Flood	0.9992	0.9997	0.9994	77007
DDoS-TCP_Flood	0.9995	0.9999	0.9997	96259
DDoS-UDP_Flood	0.9998	0.9997	0.9998	115833
DDoS-UDP_Fragmentation	0.9998	0.9998	0.9998	6141
DNS_Spoofing	0.7635	0.7916	0.7773	3829
DictionaryBruteForce	0.8359	0.5842	0.6878	279
DoS-HTTP_Flood	0.9967	0.9967	0.9967	1538
DoS-SYN_Flood	0.9994	0.9994	0.9994	43421
DoS-TCP_Flood	0.9997	0.9991	0.9994	57174
DoS-UDP_Flood	0.9996	0.9996	0.9996	71024
MITM-ArpSpoofing	0.9126	0.8665	0.8890	6583
Mirai-greeth_flood	0.9995	0.9991	0.9993	21228
Mirai-greip_flood	0.9989	0.9993	0.9991	16087
Mirai-udpplain	0.9998	0.9998	0.9998	19060
Recon-HostDiscovery	0.8342	0.8745	0.8538	2876
Recon-OSScan	0.7431	0.6824	0.7115	2103
Recon-PingSweep	0.8014	0.5650	0.6628	200
Recon-PortScan	0.7094	0.7223	0.7158	1761
SqlInjection	0.8512	0.5150	0.6417	200
Uploading_Attack	0.8390	0.4950	0.6226	200
VulnerabilityScan	0.9938	0.9975	0.9956	800
XSS	0.7284	0.5900	0.6519	200
accuracy			0.9948	1000000
macro avg	0.9234	0.8638	0.8876	1000000
weighted avg	0.9948	0.9948	0.9948	1000000

Fig. 6. Classification report and confusion matrix of XGBoost for multi classification.

and macro-F1 were achieved by XGBoost (Accuracy = 0.9961, Macro-F1 = 0.9952), with an average inference latency of 0.355 µs/sample. Full classification reports and confusion matrices are provided for XGBoost in Fig. 5 and for the remaining models in the Supplementary Material.

The confusion matrix (Fig. 5) indicates that while false positives were minimal (333 samples), the model yielded 2,765 false negatives (attacks misclassified as benign). A detailed interpretation of these misclassifications in relation to specific attack types is provided in the Discussion (Sect. 5.1).

Multi-class classification results (34 classes)

Table 5 reports held-out test performance for the 34-class setting, together with 5-fold CV macro-F1 (mean ± std), average inference latency, and training time. XGBoost achieved Accuracy = 0.9948 and Macro-F1 = 0.8876 with inference latency of 3.528 µs/sample. CatBoost achieved Accuracy = 0.9873 and Macro-F1 = 0.7405 with inference latency 1.121 µs/sample. LightGBM achieved Accuracy = 0.9879 and Macro-F1 = 0.7453 with inference latency of 3.385 µs/sample. The classification report and confusion matrix for XGBoost are shown in Fig. 6; additional matrices are provided in the Supplementary Material.

Performance on minority classes

While the XGBoost model achieved a high weighted-F1 score of 0.9947 across all 34 classes, a granular analysis reveals performance disparities for minority classes. Table 6 details the performance metrics for attack categories with the minimum sample support ($N \approx 200$ in the test set). Unlike volumetric attacks (e.g., DDoS-ICMP_Flood) which maintained F1-scores near 1.0, low-volume attacks such as Uploading_Attack, SqlInjection, and Recon-PingSweep exhibited lower detection rates, with F1-scores ranging from 0.62 to 0.72. The confusion matrix (Fig. 6) indicates that misclassifications for these categories occurred primarily towards the BenignTraffic class.

Multi-class classification results (8 classes)

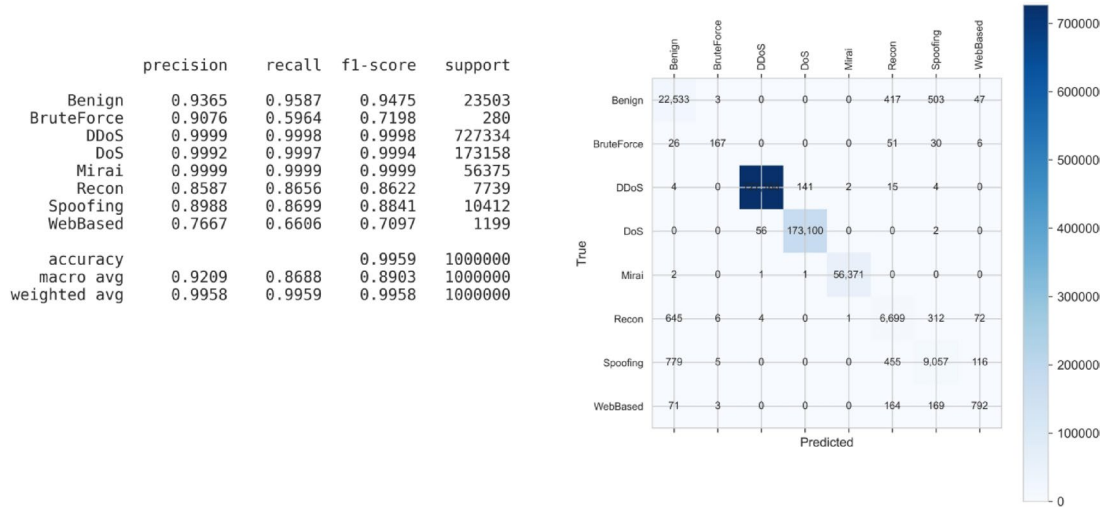
Table 7 reports held-out test performance for the 8-class (attack-family) setting, together with 5-fold CV macro-F1 (mean ± std), average inference latency, and training time. XGBoost achieved Accuracy = 0.9959 and Macro-F1 = 0.8903 with inference latency of 1.046 µs/sample. LightGBM achieved Accuracy = 0.9938 and Macro-F1 = 0.7991 with inference latency 15.897 µs/sample. CatBoost achieved Accuracy = 0.9920 and

Model	Category	Precision	Recall	F1-score
Uploading_Attack	Web-based	0.84	0.50	0.62
SqlInjection	Web-based	0.85	0.52	0.64
XSS	Web-based	0.73	0.59	0.65
Backdoor_Malware	Web-based	0.75	0.59	0.66
Recon-PingSweep	Recon	0.80	0.57	0.66
CommandInjection	Web-based	0.87	0.55	0.67
DictionaryBruteForce	BruteForce	0.84	0.58	0.69
Recon-OSScan	Recon	0.74	0.68	0.71
BrowserHijacking	Web-based	0.86	0.62	0.72

Table 6. Performance breakdown for minority classes (XGBoost, 34-Class Task).

Model	Accuracy	Macro-F1	Weighted-F1	CV Macro-F1	Inf (µs)	Train (s)
XGBoost	0.9959	0.8903	0.9958	0.8873 ± 0.0032	1.046	120.7
LightGBM	0.9938	0.7991	0.9941	0.7397 ± 0.1217	15.897	115.4
CatBoost	0.9920	0.7692	0.9923	0.7779 ± 0.0083	0.515	55.8

Table 7. Performance summary of multi-class classification models (8 classes).



	precision	recall	f1-score	support
Benign	0.9365	0.9587	0.9475	23503
BruteForce	0.9076	0.5964	0.7198	280
DDoS	0.9999	0.9998	0.9998	727334
DoS	0.9992	0.9997	0.9994	173158
Mirai	0.9999	0.9999	0.9999	56375
Recon	0.8587	0.8656	0.8622	7739
Spoofing	0.8988	0.8699	0.8841	10412
WebBased	0.7667	0.6606	0.7097	1199
accuracy			0.9959	1000000
macro avg	0.9209	0.8688	0.8903	1000000
weighted avg	0.9958	0.9959	0.9958	1000000

Fig. 7. Classification report and confusion matrix of XGBoost for 8-class classification.

Model	Binary (top features)	8-class (top features)	34-class (top features)
LightGBM	rst_count, IAT, flow_duration, Variance, Number	Covariance, urg_count, IAT, Header_Length, Magnitue	Rate, flow_duration, IAT, Header_Length, Radius
XGBoost	rst_count, Variance, Number, Weight, ICMP	IAT, Min, Magnitue, Weight, Tot size	fin_flag_number, psh_flag_number, syn_flag_number, ICMP, Min

Table 8. Top-ranked features by task (illustrative from XGBoost/LightGBM importance).

Macro-F1 = 0.7692 with inference latency of 0.515 µs/sample. The classification report and confusion matrix for XGBoost are shown in Fig. 7; remaining outputs are provided in the Supplementary Material.

Feature importance analysis

To characterize feature contributions under each task, we report gain-based feature-importance rankings for both LightGBM and XGBoost across the binary, 8-class, and 34-class settings. The importance is summarized in Table 8 and visualized in Fig. 8. Across tasks, the set of top-ranked features differs by label granularity, and the relative ordering varies between the two boosting frameworks.

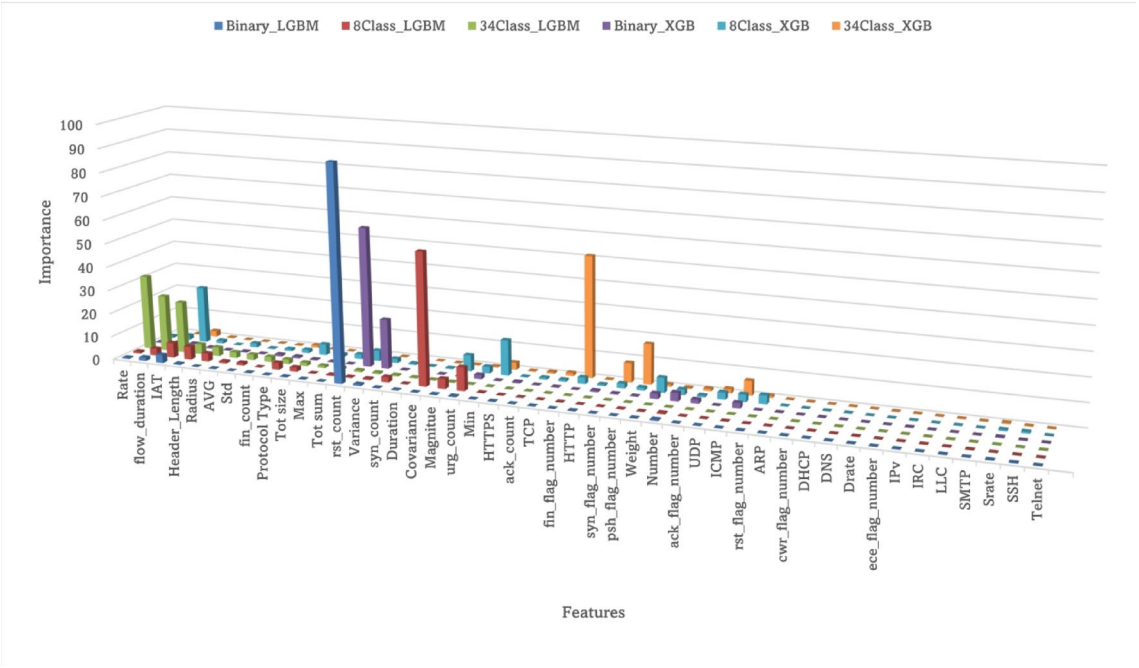


Fig. 8. Features importance across three tasks (LightGBM vs. XGBoost).

Task	#Features	CV Macro-F1 (mean ± std)	Test accuracy	Test Weighted-F1	Test Macro-F1	Training time (s)	Inference (µs/sample)
Binary	46	0.995056 ± 0.000079	0.996126	0.996133	0.995154	29.60	0.719
Binary	23	0.995028 ± 0.000076	0.996128	0.996134	0.995156	19.93	0.355
8-class	46	0.893330 ± 0.003702	0.995975	0.995943	0.895624	180.74	1.506
8-class	23	0.887294 ± 0.003211	0.995887	0.995849	0.890307	120.68	1.046
34-class	46	0.888382 ± 0.001660	0.994980	0.994861	0.887955	728.13	5.533
34-class	23	0.888262 ± 0.001327	0.994845	0.994765	0.887564	433.89	3.528

Table 9. Ablation results of gain-based feature selection using XGBoost (46 vs. 23 features).

Task	Δ Accuracy (pp)	Δ Macro-F1	Training time reduction	Inference reduction
Binary	+0.0002	+0.000002	32.68%	50.62%
8-class	−0.0088	−0.005317	33.24%	30.56%
34-class	−0.0135	−0.000391	40.41%	36.24%

Table 10. Relative effect of reducing features from 46 to 23 (23-feature model vs. 46-feature baseline).

In the binary task, highly ranked features include rst_count, variance, IAT, and flow_duration (Table 8). In the 8-class task, highly ranked features include IAT, Min, Magnitude, Weight, and Tot size (Table 8). In the 34-class task, LightGBM assigns high importance to Rate, flow_duration, IAT, Header_Length, and Radius, while XGBoost assigns high importance to TCP-flag and protocol-related features including fin_flag_number, psh_flag_number, syn_flag_number, and ICMP (Table 8).

Figure 8 also shows that multiple features receive near-zero gain in one or more task–model combinations, whereas a smaller subset concentrates most of the total gain mass.

Ablation study

We performed an ablation experiment to quantify the isolated effect of the gain-based feature selection component. XGBoost was trained and evaluated under two conditions: (i) using the full set of 46 CICIoT2023 features and (ii) using the reduced 23-feature subset selected by the LightGBM median-gain rule. The experiment was repeated for the binary, 8-class, and 34-class tasks under identical stratified sampling and five-fold cross-validation protocols.

As reported in Table 9, reducing the feature space from 46 to 23 decreased inference latency by 30–51% and training time by 33–40% across tasks. Changes in test accuracy were within ±0.014% points (Table 10).

Macro-F1 changes were small; the largest decrease was observed in the 8-class setting (Δ Macro-F1 = – 0.005317), as reported in Table 10.

Discussion and comparative analysis

This study provides a systematic evaluation of optimized gradient-boosting models for IoT intrusion detection on the CICIoT2023 benchmark. By combining balanced sampling, gain-based feature selection, and cross-validated training, the proposed pipeline achieves strong detection performance across 2-class, 8-class, and 34-class settings while maintaining microsecond-level inference latency. These results indicate that compact tree-based ensembles are practical candidates for deployment at IoT gateways where fast inference and model interpretability are important.

Analysis of minority class performance and false negatives

The experimental results highlighted a performance gap between volumetric and non-volumetric attacks. In the binary task, the 2,765 false negatives (Fig. 5) align with the specific minority classes identified in the 34-class analysis (Table 6). For instance, classes such as Recon-PingSweep and Uploading_Attack were frequently misclassified as BenignTraffic. These empirical findings tightly align with the theoretical conceptualization of IoT network behavior. IoT devices predominantly engage in deterministic, machine-to-machine (M2M) communications, establishing a highly periodic statistical baseline for features such as IAT and Flow Duration. Volumetric attacks fundamentally violate this theoretical baseline, making them easily distinguishable. In contrast, the degradation in performance for semantic and low-volume attacks occurs because these threats exploit the application layer without disrupting the network's statistical periodicity. This highlights a theoretical boundary of flow-based IDS: when attack patterns mimic legitimate human-like interactivity rather than violating the automated M2M traffic periodicity, statistical flow features alone become insufficient, necessitating supplementary Deep Packet Inspection (DPI).

Sufficiency of sample size

Regarding the undersampling strategy, the minimum threshold of $M_{min} = 1{,}000$ samples per class was strictly dictated by the intrinsic scarcity of specific attacks in the original CICIoT2023 dataset. As detailed in the distribution analysis (Fig. 2), the original sample counts for classes such as *Uploading_Attack* $N_{orig} = 1{,}252$, *Recon-PingSweep* $N_{orig} = 2{,}262$, and *Backdoor_Malware* $N_{orig} = 3{,}218$ were naturally low. Consequently, our strategy for these minority classes was not aggressive undersampling, but rather maximum data retention (e.g., retaining $\approx 80\%$ of all available *Uploading_Attack* instances). Setting M_{min} significantly higher would have necessitated synthetic data generation (e.g., SMOTE), which was avoided to preserve the integrity of the real-world traffic evaluation. The results (Table 6) indicate that this threshold was sufficient to learn discriminative patterns (F1-scores 0.62–0.72) despite the extreme imbalance against the volumetric classes.

The impact of feature selection on performance and efficiency

Feature selection is central to reducing deployment cost and achieving real-time operation. Our ablation study (Sect. 4.6, Tables 9 and 10) empirically demonstrates that the gain-based selection of a 23-feature subset is not merely a preprocessing step but a critical component for enabling efficient deployment. The selected subset effectively captures the essential discriminative information, rendering approximately half of the original features computationally redundant for the studied models.

The efficiency gains are substantial: inference latency was reduced by 30–51% and training time by 33–40%. This is exemplified by the binary task latency dropping from 0.719 µs to 0.355 µs per sample—a critical improvement for resource-constrained IoT edge devices. Crucially, these gains were achieved with a negligible impact on predictive accuracy (changes $< \pm 0.014\%$ points), confirming a highly favorable accuracy–efficiency trade-off. This outcome highlights the necessity of reporting both accuracy-centric and efficiency-centric metrics when proposing practical IoT IDS solutions.

An important consideration is the model dependence of feature importance. As shown in Fig. 8; Table 8, the specific importance rankings and the resultant optimal feature subset can vary between LightGBM and XGBoost. This sensitivity suggests that for a production system, the feature selection process should ideally be derived from, or validated against, the specific final model intended for deployment to ensure optimal compactness and efficiency.

Interpretation of feature importance and model robustness

The analysis of feature importance (Sect. 4.5, Fig. 8) reveals that the salient signals for intrusion detection are task-dependent. Binary classification prioritizes indicators of connection instability (e.g., rst_count, Variance), while finer-grained classification relies more on protocol-specific semantics (e.g., TCP flags for 34-class). This progression suggests that models naturally adapt their focus from detecting general anomalies to distinguishing specific attack signatures as the classification task becomes more detailed.

The concentration of predictive power in a consistent subset of features across tasks, coupled with the large number of features with near-zero importance, provides a principled justification for our dimensionality reduction strategy. It indicates robustness and suggests that the selected features represent generalizable flow characteristics fundamental to IoT attack detection, rather than noise tailored to the specific dataset.

Performance benchmarking and comparative analysis

When compared with related work on the CICIoT2023 dataset (Table 1), our contribution lies in the unified multi-granularity evaluation and the explicit quantification of the inference efficiency trade-off. The consistent macro-F1 advantage of XGBoost in our experiments positions it as a strong default choice when

minority-class detection is prioritized. However, we emphasize that the absolute latency values are hardware- and implementation-dependent; thus, the reported trade-offs should be interpreted within the context of our experimental setup. The primary finding is the demonstrated feasibility of achieving high accuracy with microsecond-level inference, a benchmark for practical deployment.

Limitations and future directions

Despite promising results, several limitations must be acknowledged. First, evaluation on a single benchmark dataset (CICIoT2023) limits claims of generalizability; performance may vary in other IoT environments or under concept drift. Second, the use of stratified undersampling creates a balanced evaluation environment but does not reflect operational attack prevalence. Future work should investigate cost-sensitive learning and calibration under realistic priors. Third, operating on flow-level metadata leaves the system potentially vulnerable to advanced adversarial evasion and payload-based attacks not evaluated here. Future research should explore streaming adaptation, cross-dataset validation, and robustness testing while preserving the low-latency constraint. Fourth, regarding assumptions and constraints on generalizability: This framework operates on the foundational assumption that the statistical flow patterns captured within the CICIoT2023 testbed accurately represent broader IoT environments. However, real-world edge computing deployments often exhibit higher stochasticity—induced by network jitter, variable latency, and highly heterogeneous device configurations—than controlled laboratory datasets. Consequently, the generalizability of the trained model to unseen, highly dynamic IoT topologies or entirely novel protocols may be constrained. Deploying this framework in production networks would likely require an initial calibration phase to adapt the feature standardization parameters to the specific statistical 'noise' of the target physical environment.

Adversarial robustness and feature selection

Finally, we acknowledge the challenge of adversarial evasion. As outlined in the threat model (Sect. 3.2), sophisticated adversaries may attempt to mask attacks via traffic shaping (e.g., manipulating packet timing or sizing). While our framework's reliance on flow-level statistics (e.g., *IAT*, *Magnitude*) renders it immune to payload obfuscation or encryption, it remains theoretically susceptible to statistical mimicry. However, the feature selection process—which reduced the input space from 46 to 23 dimensions—inadvertently enhances practical robustness. By pruning brittle, easily modifiable attributes (specifically boolean protocol flags like *HTTP* and *DNS*) and retaining complex distributional metrics (e.g., *Variance*, *Covariance*, *Radius*), the model minimizes its reliance on features that are trivial for an adversary to spoof. To evade the retained feature set, an attacker must manipulate high-order traffic statistics, which typically incurs a higher operational cost and may degrade the attack's potency (e.g., reducing the packet rate to match benign *IAT* profiles).

Threats to validity

A structured analysis of potential threats to the validity of our findings is presented below.

Internal validity

The primary concern for internal validity relates to the potential for information leakage during data preprocessing. To mitigate this, a strict sequential protocol was employed: the dataset was first split into stratified training and testing subsets; subsequently, all feature selection and standardization procedures were fitted exclusively on the training data before being applied to the test set. This design ensures that the test data did not influence the construction of the model's input representation. While this protocol minimizes optimistic bias, a nested cross-validation approach integrating feature selection within each training fold could offer an even more robust evaluation in future work.

External validity

The generalizability of our conclusions is constrained by several factors. First, the evaluation is based entirely on the CICIoT2023 benchmark. Its specific composition of IoT devices, attacks, and feature engineering may not fully represent other network environments, limiting cross-dataset applicability. Second, the use of stratified undersampling to create a balanced dataset, while methodologically necessary for handling extreme class imbalance and computational constraints, alters the natural attack prevalence. Consequently, performance in operational settings with highly skewed class distributions may vary. Finally, the study is limited to flow-level metadata; attacks relying on packet payload manipulation or advanced adversarial patterns were not within the scope of this evaluation.

Construct validity

We endeavored to ensure construct validity by reporting multiple performance metrics (accuracy, macro-F1, weighted-F1) and explicit measurements of computational efficiency (training time, inference latency). However, two key considerations persist. First, the reported inference latency is contingent upon the specific hardware (e.g., GPU acceleration), software libraries, and measurement protocol used. The absolute values should therefore be interpreted as demonstrating relative efficiency gains within our experimental setup rather than as universal benchmarks. Second, feature importance rankings and the resulting selected subset are model-dependent. As our analysis shows, different gradient boosting architectures (e.g., XGBoost vs. LightGBM) can assign divergent importance to the same features. Therefore, the identified optimal feature set is intrinsically linked to the selection algorithm used and may not generalize perfectly to other model families.

Conclusion validity

Threats to conclusion validity concern the robustness of the inferences drawn from the experimental design. Although we utilized stratified 5-fold cross-validation and grid search for hyperparameter tuning, the exploration was necessarily finite. More extensive hyperparameter optimization or alternative search strategies might yield different performance trade-offs. Furthermore, the core comparative findings are most definitive within the gradient boosting model family evaluated. While baseline models were included, the principal results regarding feature selection efficacy and microsecond-level latency are most directly applicable to this class of algorithms, which are known to excel on tabular data.

Conclusion

This study presented a comprehensive evaluation of optimized gradient-boosting models for IoT intrusion detection, demonstrating that frameworks like XGBoost can achieve high detection accuracy (exceeding 99.4%) across binary, 8-class attack-family classification, and fine-grained (34-class) classification tasks while maintaining microsecond-level inference latency. A critical finding is that gain-based feature selection can reduce the input dimensionality by approximately 50%, yielding a 30–50% improvement in inference speed with a negligible impact on accuracy, thereby confirming a highly favorable efficiency–accuracy trade-off for resource-constrained IoT gateways.

These compelling results are subject to important limitations. First, the evaluation is confined to a single benchmark dataset (CICIoT2023), which may not encompass the full heterogeneity of real-world IoT traffic. Second, the use of stratified undersampling for class balancing creates an artificial evaluation environment that does not reflect the highly imbalanced attack prevalence typical of operational networks. Third, the practical efficiency gains from feature selection are partially model-dependent, as importance rankings vary across gradient boosting implementations. Finally, the study operates on flow-level statistics and does not assess robustness against adversarial evasion or payload-based attacks.

Future work should therefore focus on validating the proposed framework's generalizability through cross-dataset evaluation and testing under realistic, streaming data conditions with inherent class imbalance. Investigating online learning techniques to handle concept drift and adversarial robustness testing would further strengthen the approach for production deployment. Extending the feature analysis to include temporal patterns and exploring hybrid models could enhance detection for sophisticated, multi-stage attacks while preserving the low-latency requirement essential for IoT edge security.

Data availability

The raw CICIoT2023 dataset analyzed in the current study is available in the Canadian Institute for Cybersecurity repository at [https://www.unb.ca/cic/datasets/iotdataset-2023.html](.)[30]. Researchers intending to replicate the full data ingestion and feature selection pipeline described in Algorithm 1 are advised to download the raw dataset from the original source.**Extended data[31]: ****Zenodo Record: ** *Supplementary material - An Optimized Gradient Boosting Framework for IoT Intrusion Detection: A Comprehensive Evaluation on the CICIoT2023 Dataset (Version 3.0.0).* This repository contains: 1. **Source Code: ** Python scripts implementing the complete experimental pipeline, including stratified undersampling, gain-based feature selection (Algorithm 1), and model evaluation for Binary, 8-class, and 34-class tasks. 2. **Processed Datasets: ** Stratified subsets of the training and testing data (with the optimized 23-feature set) used for the reported ablation studies.

Glossary

IoT	Internet of things
IDS	Intrusion detection system
CICIoT2023	Canadian Institute for Cybersecurity-IoT2023
DDoS	Distributed denial-of-service
DoS	Denial-of-service
DNN	Deep neural network
RNN	Recurrent neural network
LSTM	Long short-term memory
CNN	Convolutional neural network
MLP	Multi-layer perceptron
KAN	Kolmogorov-Arnold networks
GB	Gradient boosting

Received: 26 November 2025; Accepted: 31 March 2026
Published online: 11 April 2026

References

1. Asharf, J. et al. A review of intrusion detection systems using machine and deep learning in Internet of Things: challenges, solutions and future directions. *Electronics* **9**, 1177 (2020).
2. C Neto, E. et al. CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* https://doi.org/10.3390/s23135941 (2023).
3. Abbas, S. et al. A novel federated edge learning approach for detecting cyberattacks in IoT infrastructures. *IEEE Access.* **11**, 112189–112198 (2023).
4. Li, Q., Liu, Y., Niu, T. & Wang, X. Improved ResNet model based on positive traffic flow for IoT anomalous traffic detection. *Electronics* **12**, 3830 (2023).

5. Sicari, S., Rizzardi, A., Grieco, L. A. & Coen-Porisini, A. Security, privacy and trust in Internet of Things: the road ahead. *Comput. Netw.* **76**, 146–164 (2015).
6. Jakotiya, K., Shirsath, V. & Mishra, R. G. Feature engineering using machine learning techniques on CIC-IoT-2023 dataset. *Mach. Vis. Augment. Intell.* 717–727 (2024).
7. Tseng, S. M., Wang, Y. Q. & Wang, Y. C. Multi-class intrusion detection based on transformer for IoT networks using CIC-IoT-2023 dataset. *Future Internet*. **16**, 284 (2024).
8. Shtayat, M. M., Hasan, M. K., Sulaiman, R., Islam, S. & Khan, A. U. R. An explainable ensemble deep learning approach for intrusion detection in industrial Internet of Things. *IEEE Access.* **11**, 115047–115061 (2023).
9. Le, T. T. H., Wardhani, R. W., Putranto, D. S. C., Jo, U. & Kim, H. Toward enhanced attack detection and explanation in intrusion detection system-based IoT environment data. *IEEE Access.* **11**, 131661–131676 (2023).
10. Bello, O., Zeadally, S. & Badra, M. Network layer inter-operation of device-to-device communication technologies in Internet of Things (IoT). *Ad Hoc Netw.* **57**, 52–62 (2017).
11. Wang, Y. C., Houng, Y. C., Chen, H. X. & Tseng, S. M. Network anomaly intrusion detection based on deep learning approach. *Sensors* **23**, 2171 (2023).
12. Jony, A. I. & Arnob, A. K. B. A long short-term memory based approach for detecting cyber attacks in IoT using CICIoT2023 dataset. *J. Edge Comput.* **3**, 28–42 (2024).
13. Jaradat, A. S., Nasayreh, A., Al-Na'amneh, Q. & Gharaibeh, H. & Al Mamlook, R. E. Genetic optimization techniques for enhancing web attacks classification in machine learning. In *Proc. IEEE DASC/PiCom/CBDCom/CyberSciTech* 130–136 (2023).
14. He, M. et al. Reinforcement learning meets network intrusion detection: a transferable and adaptable framework for anomaly behavior identification. *IEEE Trans. Netw. Serv. Manag.* **21**, 2477–2492 (2024).
15. Amouri, A., Al Rahhal, M. M., Bazi, Y., Butun, I. & Mahgoub, I. Enhancing intrusion detection in IoT environments: an advanced ensemble approach using Kolmogorov–Arnold networks. In *Proc. 2024 International Symposium on Networks, Computers and Communications (ISNCC)* 1–6 (2024).
16. Adewole, K. S., Jacobsson, A. & Davidsson, P. Intrusion detection framework for Internet of Things with rule induction for model explanation. *Sensors* **25**, 1845 (2025).
17. Boahen, E. K. et al. Cross-platform intrusion detection in online social networks. *IEEE Transactions on Services Computing*. https://ieeexplore.ieee.org/document/11186216 (IEEE, 2025).
18. Boahen, E. K., Sosu, R. N. A., Ocansey, S. K., Xu, Q. & Wang, C. A. S. R. L. Adaptive swarm reinforcement learning for enhanced OSN intrusion detection. *IEEE Transactions on Information Forensics and Security*. https://ieeexplore.ieee.org/document/10741305 (IEEE, 2024).
19. He, H. & Garcia, E. A. Learning from imbalanced data. *IEEE Trans. Knowl. Data Eng.* **21**, 1263–1284 (2009).
20. Han, J., Kamber, M. & Pei, J. *Data Mining: Concepts and Techniques* 3rd edn (Elsevier, 2012).
21. Ke, G. et al. LightGBM: a highly efficient gradient boosting decision tree. *Adv. Neural Inf. Process. Syst.* **30**, 3149–3157 (2017).
22. Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. SMOTE: synthetic minority over-sampling technique. *J. Artif. Intell. Res.* **16**, 321–357 (2002).
23. Bergstra, J. & Bengio, Y. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.* **13**, 281–305 (2012).
24. Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* 3rd edn (O'Reilly Media, 2022).
25. Kohavi, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)* **14**, 1137–1145 (1995).
26. Witten, I. H., Frank, E., Hall, M. A. & Pal, C. J. *Data Mining: Practical Machine Learning Tools and Techniques* 4th edn (Morgan Kaufmann, 2016).
27. Pedregosa, F. et al. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* **12**, 2825–2830 (2011).
28. Sokolova, M. & Lapalme, G. A systematic analysis of performance measures for classification tasks. *Inf. Process. Manag.* **45**, 427–437 (2009).
29. Krizhevsky, A., Sutskever, I. & Hinton, G. E. ImageNet classification with deep convolutional neural networks. *Commun. ACM*. **60**, 84–90 (2017).
30. Canadian Institute for Cybersecurity. CIC IoT dataset 2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. https://www.unb.ca/cic/datasets/iotdataset-2023.html (Canadian Institute for Cybersecurity, 2023).
31. Almahaqeri, S. et al. Supplementary material – An optimized gradient boosting framework for IoT intrusion detection: a comprehensive evaluation on the CICIoT2023 dataset (v3.0.0). *Zenodo* https://doi.org/10.5281/zenodo.17070208 (2026).

Author contributions

Salah Almahaqeri: Conceptualization, methodology, formal analysis, data curation, visualization, and writing—original draft preparation; Mohammed Hashem Almourish: Conceptualization, formal analysis, supervision, writing—review and editing; Adel A. Nasser: Conceptualization, methodology, formal analysis, software, and writing—original draft preparation, writing—review and editing; Abed Saif Ahmed Alghawli: methodology, formal analysis, funding acquisition, writing—original draft preparation, and writing—review and editing; Amani A. K. Elsayed: validation, investigation, resources, writing—review and editing; and Ali.N.Alhejoj: formal analysis, writing—review and editing.

Funding

This study was supported via funding from Deanship of Scientific Research, Prince Sattam bin Abdulaziz University [project number: PSAU/2025/R/1446].

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to A.A.N.